

본산 시스템과 장시간 비동기 작업이 많은 AI/3D 도메인에서 서비스를 런칭하고 안정화해온 백엔드 엔지니어입니다. 3D방구미기에서 상품 검색·이미지 합성·스타일 추천을 제공하는 멀티에이전트 기반 AI 인터리어 비서를 설계·구축하고 운영했습니다. LLM의 비결정성을 통제하기 위한 constrained orchestration과 runtime guardrails를 설계하고, 비동기 처리·SSE streaming·provider 장애 격리를 적용했습니다. 글로벌 AI 서비스 런칭과 수익화, 3D 자동화 파이프라인 구축을 통해 서비스 안정성과 비즈니스 성과를 함께 만들어왔습니다.

📄 경력 사항

Bucketplace (오늘의집)

📅 2022년 8월 - 현재

Senior Backend Engineer

- **AI Agent Orchestration:** 3D방구미기의 상품 검색·이미지 합성·스타일 추천 등 다양한 AI 기능을 하나의 흐름으로 통합하는 오케스트레이터를 설계·구축하고 운영했습니다. LLM에는 사용자 의도 해석과 도구 파라미터 생성을 맡기되, 순서 의존성이 있는 tool chain과 필수 도구 호출은 코드로 강제하는 constrained orchestration을 설계했습니다.
 - 이미지 합성 등 장시간 AI 작업에서 클라이언트 연결 단절에 대응하기 위해서 요청을 비동기로 처리하고, 상태와 중간 결과를 저장했으며, SSE 재연결 시 Redis Stream 이벤트를 replay해 결과를 이어받을 수 있도록 했습니다.
 - 특정 LLM provider의 장애가 서비스 장애로 이어지지 않도록 retry, circuit breaker, provider fallback을 적용했습니다. Circuit breaker 상태를 service-model-API key 단위로 관리해서 문제가 발생한 대상만 차단하고, 개별 key의 quota 초과에도 대응했습니다.
 - Production trace sample과 기능 요구사항을 바탕으로 기능과 사용자 의도별 coverage matrix를 설계하고, 도구 호출과 파라미터·응답·멀티턴 흐름을 검증하는 **85개 시나리오**의 trajectory regression suite를 구축했습니다. 운영 응답의 **94%**가 품질 기준을 충족함을 확인했습니다.
- **Ohouse AI 런칭:** 6주 목표를 **4주로 앞당겨 170개국** 글로벌 런칭을 완료했습니다. 사용자 요청 API와 장시간 AI 작업을 분리하는 메시지 큐 기반 워커를 구축하고, 오토스케일링·동시성 제한·재시도·장애 격리를 적용해 예측하기 어려운 트래픽을 안정적으로 처리했습니다.
- **Ohouse AI 수익화/고도화:** Apple-Google IAP lifecycle을 하나의 상태머신으로 추상화해서 구독 시스템을 구축하고 결제 정합성을 확보했습니다. Amazon Affiliate 정책을 반영한 RAG 기반 상품 추천으로 **CVR 10.5%**를 달성해 수익 모델을 검증했습니다.
 - 특정 LLM provider의 장애가 서비스 장애로 이어지지 않도록 provider fallback과 circuit breaker를 적용했습니다. Spring 기반 LLM framework의 제약을 해소하기 위해 Python·LangGraph 기반으로 재설계해서 AI workflow를 유연하게 확장할 수 있도록 했습니다.
- **3D방구미기 성능·안정성:** API 응답 시간을 **200ms→80ms**로 단축하고 payload를 **54% 절감**했으며, 모델 로딩 시간을 **10초→2.5초**로 줄였습니다. 부하 테스트로 이벤트·챌린지의 트래픽 증가를 사전 검증해 **10.2배 트래픽**에서 **99.95% 가용성**을 유지했습니다.
 - 성능·안정성 개선을 바탕으로 **WAU 704% 성장**과 **연간 GMV 600%(8.6억)** 성장에 기여했습니다.
- **3D 에셋 자동화:** 수작업 3D 제작 파이프라인을 자동화해 월 생산량 **68개→800개**, 비용 **88% 절감**을 달성해 **Eng Award**를 받았습니다. 3D 에셋 최적화 기법을 개선해 GPU 사용량을 **20% 감소**시키고 파일 크기를 **51% 축소**했으며, draw call을 **3,000→20**으로 줄였습니다. 에셋 관리 시스템을 구축해 연간 **40일 이상**의 운영 시간을 절감했습니다.
- **AR 경험 개선:** AR 환경에 LoD 자동 생성을 적용해 3D 모델을 **90% 경량화**했습니다. AR Size Box를 구축해 AR 지원 상품을 **2,210개→231,000개(104배)**로 확대하고 구매 전환율 **1.7배** 향상에 기여했습니다.
- **기술 리더십:** 신규 입사자와 FE·ML 엔지니어가 백엔드에 독립적으로 기여할 수 있도록 온보딩과 코드 리뷰를 주도했습니다. 릴리즈 노트와 데이터 분석을 자동화해 팀의 배포 주기를 단축하고 **AI Award, AI Native MVP**를 받았습니다.

LG CNS

📅 2020년 1월 - 2022년 7월

System & Solution Programming Consultant

- Node.js · Redis 기반 실시간 싱크 서버를 설계·운영하고, WebSocket 기반 다중 사용자 동시 편집 환경을 구축했습니다.
- GitLab/Jenkins 기반 CI/CD 파이프라인을 자동화해 빌드·테스트·배포 리드타임을 단축하고, Unity 클라이언트와 연동되는 백엔드 API를 개발했습니다.

🎓 학력 사항

서울시립대학교 | 석사, 컴퓨터과학 | 인공지능 시스템의 성능 개선 연구 / 학·석사 연계 프로그램을 통해 석사 학위 조기 취득

📅 2018년 - 2019년

서울시립대학교 | 학사, 컴퓨터과학부

📅 2012년 - 2017년

AI Agent Orchestration과 Platform 확장

문제

유저 리서치를 통해 상품 탐색 과정과 모바일 3D 조작 난이도가 3D방꾸미기의 주요 진입 장벽임을 파악했습니다. 이를 해소하기 위해 멀티에이전트 기반 AI 인터리어 비서 서비스를 구축했습니다. 상품 검색·이미지 합성·스타일 추천 등 다양한 AI 기능을 하나의 사용자 흐름으로 제공하면서, LLM의 비결정성과 외부 AI 서비스 장애를 통제해야 했습니다.

동시에 조직 내 여러 팀이 에이전트 서비스를 구축하고 있어, RP에서 구현한 Agent 아키텍처를 공통 플랫폼으로 확장할 필요가 있었습니다.

접근

A2A 프로토콜을 기반으로 에이전트 간 인터페이스 계약을 정의하고, 사용자 요청을 해석해 여러 에이전트의 실행을 제어하는 오케스트레이터를 구축했습니다. LLM에는 사용자 의도 해석과 도구 파라미터 생성을 맡기되, 순서 의존성이 있는 tool chain과 필수 도구 호출은 코드로 강제하는 programmatic enforcement를 적용했습니다.

실행 상태에 따라 다음 action을 지정하고, 긴 UUID를 실행 컨텍스트 범위의 짧은 ID로 변환·검증하는 runtime guardrails를 구축했습니다. 이미지와 파일은 ID로 참조하고 필요할 때만 불러오는 lazy hydration을 적용해서 멀티모달 컨텍스트의 크기를 제어했습니다.

장시간 AI 작업은 비동기로 처리하고 실행 상태와 중간 결과를 저장했으며, SSE 재연결 시 Redis Stream의 이벤트를 replay해서 결과를 이어받을 수 있도록 했습니다. 외부 LLM 호출에는 retry-circuit breaker-provider fallback을 적용하고, 하위 에이전트의 오류 케이스별로 graceful degradation을 적용했습니다.

RP에서 검증한 Agent 아키텍처를 공통 플랫폼으로 확장하면서 execution runtime core와 session service, SSE endpoint의 설계·구현을 담당했습니다.

SDK-agnostic canonical Engine Event를 정의하고 ADK event를 변환하는 adapter를 구현해 SDK별 차이를 격리했습니다.

결과

멀티에이전트 기반 AI 인터리어 비서를 3D방꾸미기 실서비스에 적용하고 운영했습니다. Production trace sample에서 운영 응답의 **94%**가 품질 기준을 충족함을 확인했습니다.

기능과 사용자 의도별 **85개 시나리오**의 trajectory regression suite를 구축해서 코드 변경에 따른 behavioral regression을 탐지할 수 있도록 했습니다.

Execution runtime을 RP 실행 경로에서 검증하고 AI Agent Platform의 공통 runtime으로 확장하고 있습니다.

Ohouse AI 런칭 및 고도화

문제

실제 인테리어 사진에 가구를 자동 배치하는 AI 서비스를, 제한된 리소스와 여러 팀 의존성이 얽힌 상황에서 6주 안에 글로벌 런칭해야 했습니다.

멀티턴 이미지 생성 특성상 사용자별 요청 순서 보장과 쿼터 제한 대응이 필수였고, 외부 AI 장애 전파도 차단해야 했습니다.

접근

메시지 큐 기반 비동기 워커로 API와 처리를 분리하고, Kubernetes 오토스케일링과 동시성 제한으로 독립 운영했습니다.

재시도·격리 정책을 설계하고, 인증 서버 준비 지연을 JWT 기반 게스트 인증으로 우회해 런칭을 앞당겼습니다.

런칭 후에는 Apple/Google IAP 상태머신과 LLM Provider Fallback + Circuit Breaker, AI provider 전환 구조를 적용하고, Spring LLM 프레임워크 한계로 Python 전환 후 graph-based 구조로 재설계해 수익화와 확장이 가능하게 했습니다.

Amazon Affiliate 정책 제약을 반영한 RAG 기반 추천 경로를 설계해 실시간 제휴 상품 추천을 제공했습니다.

결과

4주 만에 170개국 런칭을 완료하고, 오토스케일링과 백프레셔로 예측 불가 트래픽에서도 장애 없이 운영했습니다.

AI provider 전환 가능한 연동 구조로 외부 장애에서 빠르게 복구했고, 제휴 상품 **CVR 10.5%**로 수익 모델을 검증했습니다.

3D방꾸미기 성능·안정성 강화

문제

모바일 3D 인테리어 배치 서비스에서 초기 로딩 지연이 사용자 이탈과 성장 정체의 주요 병목이었습니다. 이벤트·챌린지의 트래픽 증가에도 안정적으로 운영해야 했습니다.

접근

API 호출 구조를 개선해 응답 시간과 payload를 줄이고, 모델 로딩 병목을 개선했습니다. 큐 기반 지표 모니터링과 안전한 종료 규칙으로 스케일링을 안정시키고, 부하 테스트로 이벤트·챌린지 시나리오를 사전 검증했습니다.

결과

API 응답 시간 **200ms→80ms**, payload **54% 절감**, 모델 로딩 **10초→2.5초**를 달성했습니다. **10.2배 트래픽**에서 **99.95% 가용성**을 유지했으며, 성능·안정성 개선을 바탕으로 **WAU 704%**, **연간 GMV 600%(8.6억)** 성장에 기여했습니다.